



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Del Toro Ortiz Juan Pablo Yasbeth

N° de Cuenta: 317031814

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 7

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 16/04/2026

CALIFICACIÓN: _____

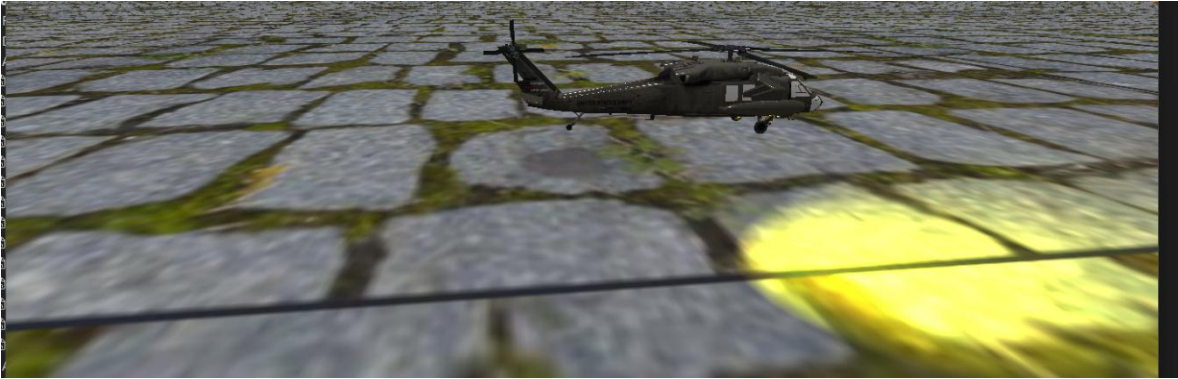
REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

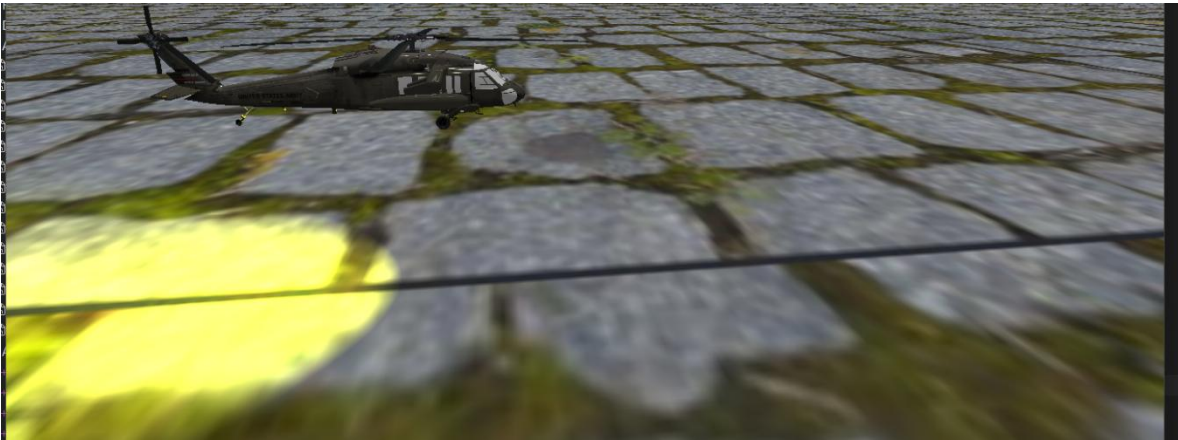
Instrucciones:

- 1. Agregar luz de tipo spotlight para el helicóptero de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight en el suelo hacia adelante y al retroceder (mover con teclado hacia dirección de X positiva) ilumine con un spotlight el suelo hacia atrás. Son dos spotlights diferentes que se prenderán y apagarán de acuerdo a la dirección en la que esté moviéndose el helicóptero.
- 2. Crear o instanciar una pecera (traslúcida) con las normales hacia adentro y en la parte superior imagen de agua e instanciar dentro de la pecera al pez abisal con movimiento de teclado para subir y bajar en diagonal.
- 3. Agregar una luz de tipo puntual de color azul ligada al bulbo del pez que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara de su práctica 7 como esta luz (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual), de tal forma que se pueda ver: las 2 luces apagadas, las 2 luces prendidas, una luz prendida y una luz apagada y viceversa.
- 4. Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del bulbo del pez y que ilumine en cualquier dirección dentro de la pecera al presionar teclas asignadas al eje x, eje y, eje z.

Capturas de pantalla Actividad 1:



Helicóptero avanzando con spotlight 1

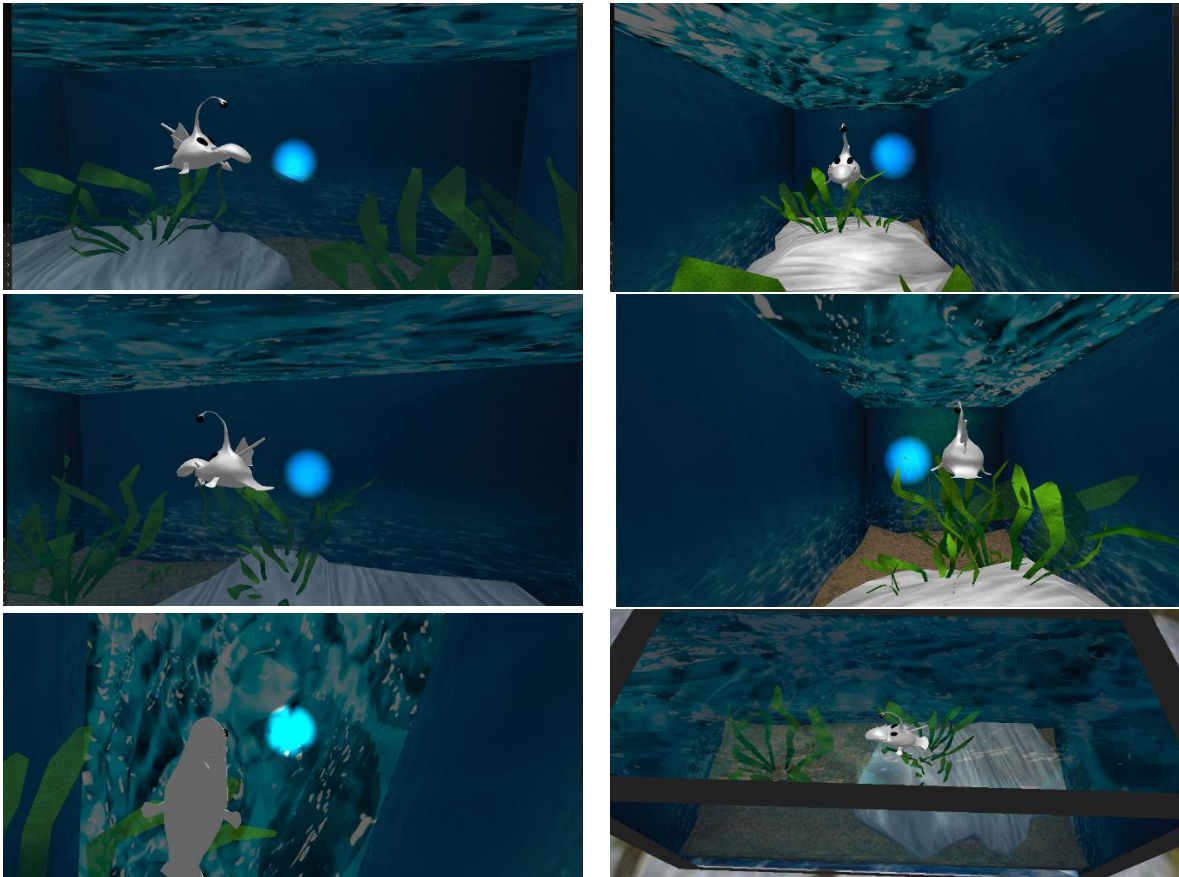


Helicóptero retrocediendo con spotlight 2

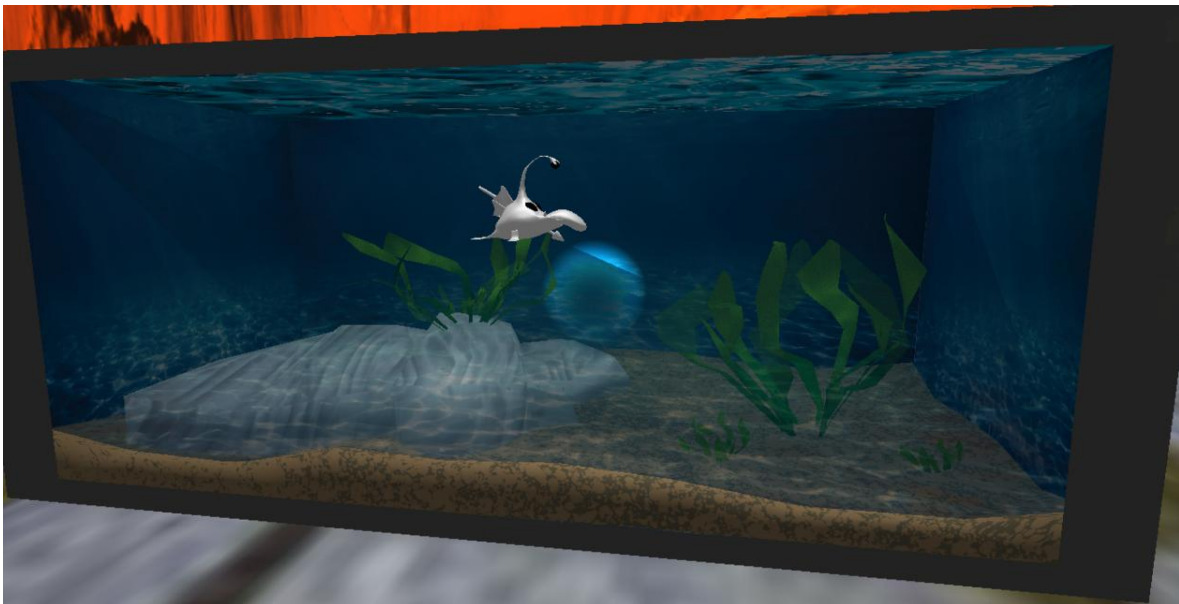
Video del funcionamiento:

<https://youtu.be/ylyYF09tAa8>

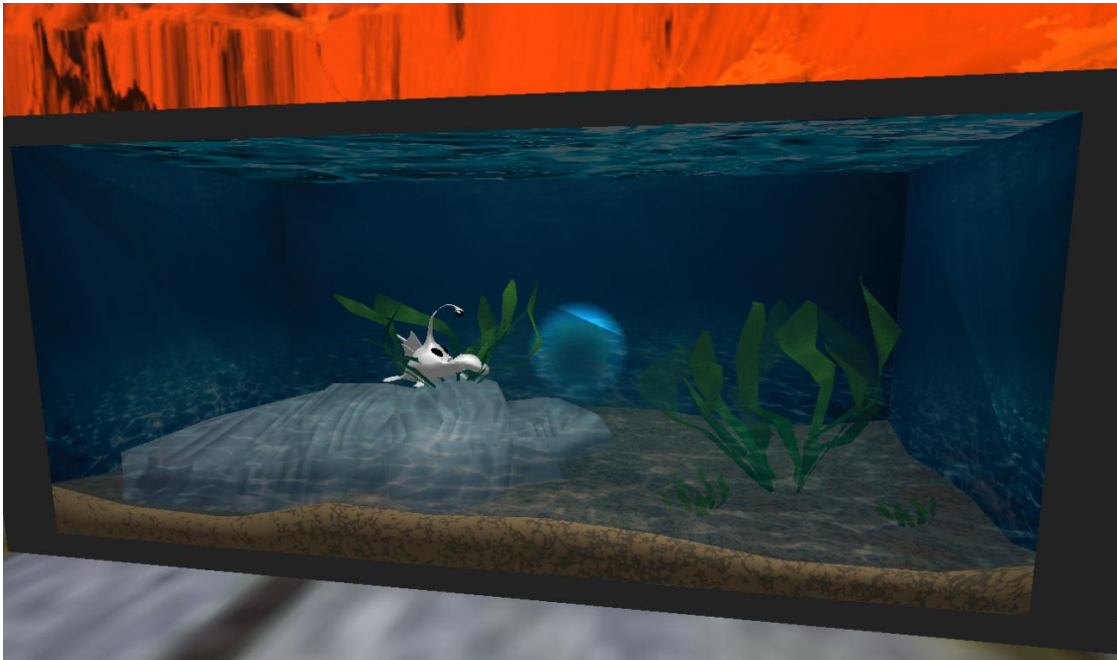
Capturas de pantalla Actividad 2:



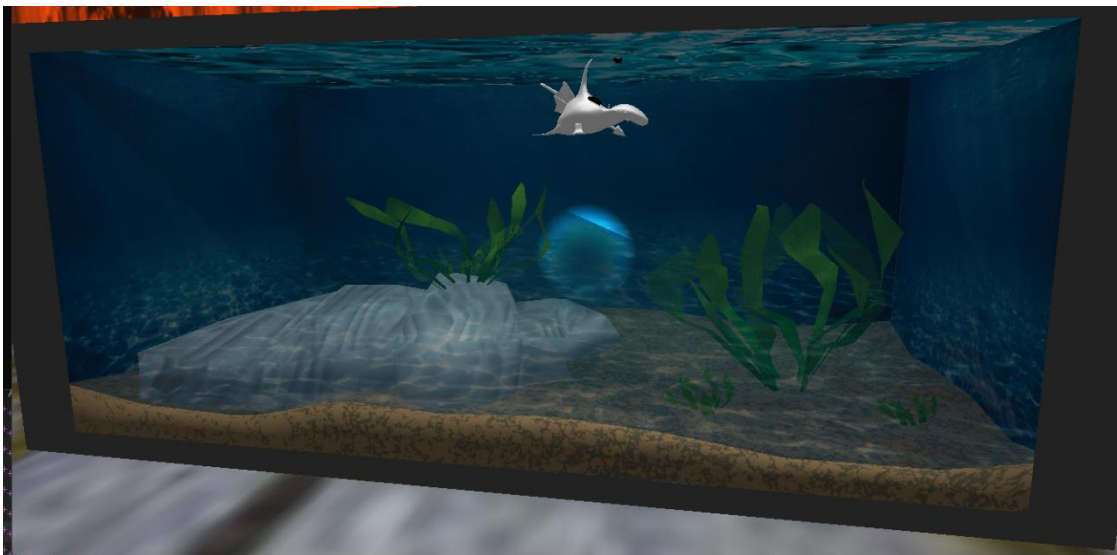
Caras con normales internas iluminadas y techo con textura diferente (agua) al resto de muros transparentes



Posición base del pez



Descenso del pez en diagonal

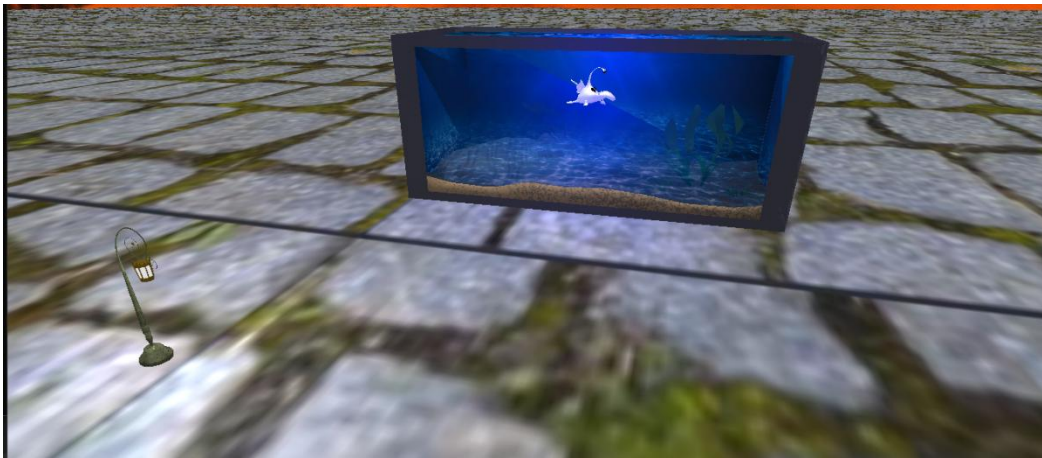


Asenso del pez en diagonal

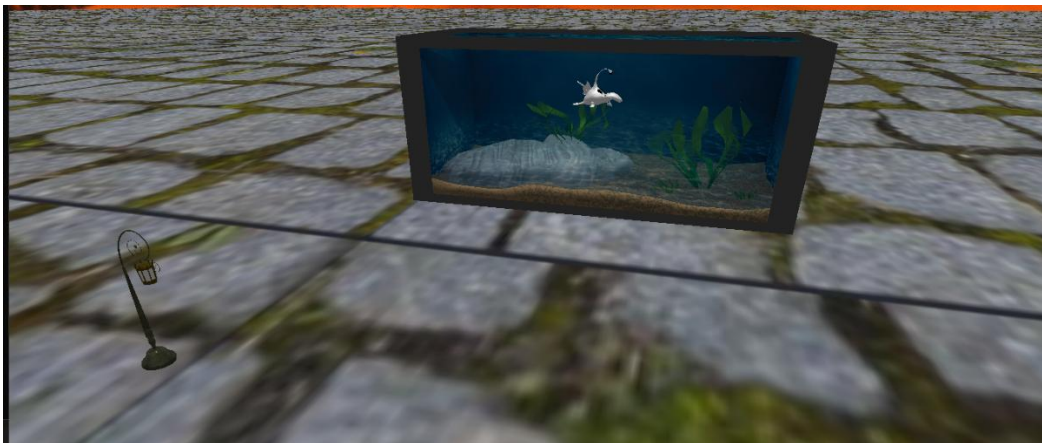
Video del funcionamiento:

<https://youtu.be/Ncvl1wV8Ab0>

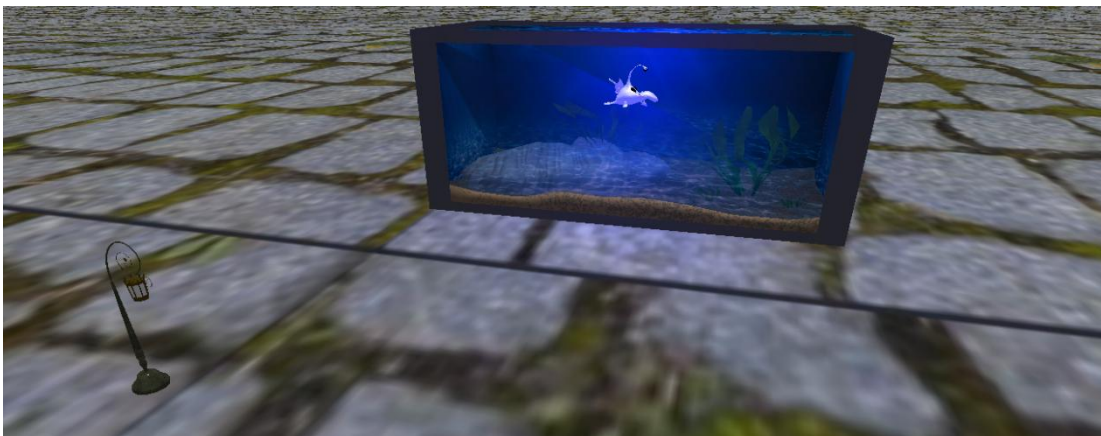
Capturas de pantalla Actividad 3:



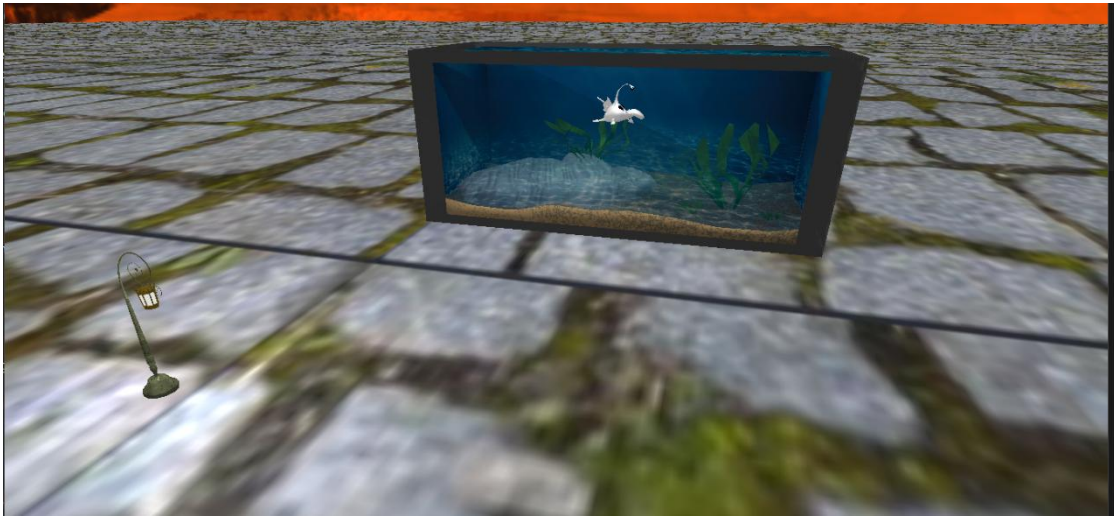
Pez y Faro iluminados



Pez y Faro apagados



Pez iluminado, Faro apagado

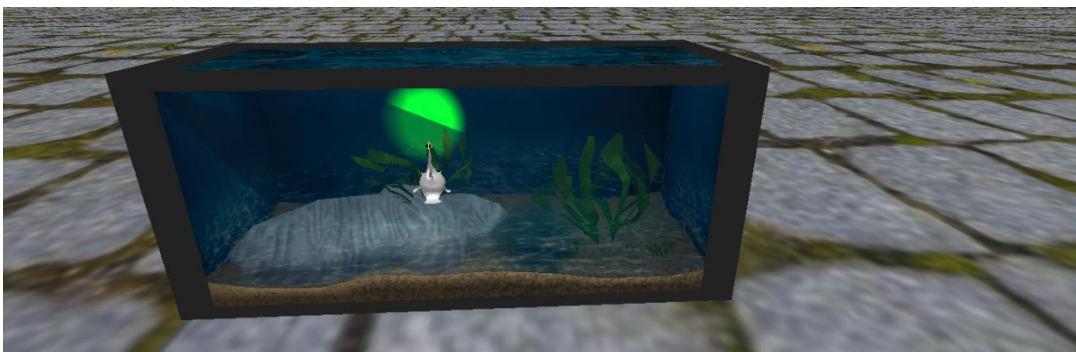


Pez apagado, Faro iluminado

Video del funcionamiento:

<https://youtu.be/mNd8E89SHdc>

Capturas de pantalla Actividad 4:





Video del funcionamiento:

<https://youtu.be/5nlfbxnEEgw>

Código generado:

Código de modificaciones en Window.h

```
25 // --- GETTER PARA CAMBIO DE LUZ ---  
26 // Cambiamos bool por int  
27 int getLuzPrendida() { return luzPrendida; }
```

```
62 // Variable para luz (Faro/Escena)  
63 // Cambiamos bool por int  
64 int luzPrendida;
```

Código de modificaciones en Window.cpp

```
38  
39 // Estado inicial  
40 //luzPrendida = true;  
41 luzPrendida = 0; // Iniciamos en el estado 0
```

```
148 // --- CICLO DE LUCES (Tecla E) ---  
149 if (key == GLFW_KEY_E && action == GLFW_PRESS)  
150 {  
151     theWindow->luzPrendida++; // Suma 1 al contador  
152  
153     // Si el contador llega a 4, lo regresamos a 0  
154     if (theWindow->luzPrendida > 3)  
155     {  
156         theWindow->luzPrendida = 0;  
157     }  
158 }
```

Código de modificación en PointLigth.h

```
17  
18 //Poder configurar posicion de la luz  
19 void SetPos(glm::vec3 pos) { position = pos; }
```

Código de los modelos:

```
50      //--- FAROL TEXTURIZADO ---
51      Model Farola_M;
52      // --- PECERA TEXTURIZADA ---
53      Model Pecera_M;
54      // -- PEZ ---
55      Model Pez_M;
```

Código de carga de modelos:

```
217      // --- CARGA DE MODELOS ---
218      Blackhawk_M = Model();
219      Blackhawk_M.LoadModel("Models/uh60.obj");
220      Farola_M = Model();
221      Farola_M.LoadModel("Models/LAP.obj");
222      Pecera_M = Model();
223      Pecera_M.LoadModel("Models/acuario.obj");
224      Pez_M = Model();
225      Pez_M.LoadModel("Models/Pez.obj");
```

Código de luces:

```
240      // --- LUZ DEL SOL (Direccional) ---
241      mainLight = DirectionalLight(1.0f, 1.0f, 1.0f,
242      0.4f, 0.4f,
243      0.0f, -1.0f, 0.0f);
244
245      // -- LUZ DE LA FAROLA (En el índice 0) --
246      pointLights[0] = PointLight(1.0f, 1.0f, 1.0f, // Color Blanco
247      0.5f, 1.0f, // IA, ID
248      0.0f, 1.5f, 5.0f, // Posición (X, Y, Z)
249      1.0f, 0.5f, 0.0f); // Atenuación
250
251      // -- LUZ DEL PEZ (En el índice 1) - Color Azul --
252      pointLights[1] = PointLight(0.0f, 0.0f, 1.0f, // Color Azul
253      0.5f, 1.0f, // IA, ID
254      0.0f, 0.0f, 0.0f, // Posición temporal
255      0.0f, 0.5f, 0.5f); // Atenuación
256
```

```

259 // --- LUZ DE LA CAMARA (Índice 0) ---
260 spotLights[0] = SpotLight(1.0f, 1.0f, 1.0f, // Color Blanco
261     0.0f, 2.0f, // IA, ID
262     0.0f, 0.0f, 0.0f, // Posición
263     0.0f, -1.0f, 0.0f, // Dirección
264     1.0f, 0.0f, 0.0f, // Atenuación
265     5.0f); // Ángulo
266 spotLightCount++;
267
268 // --- LUZ HELICÓPTERO ADELANTE (Índice 1) ---
269 spotLights[1] = SpotLight(1.0f, 1.0f, 0.0f, // Color Amarillo
270     0.0f, 2.0f, // IA, ID
271     0.0f, 0.0f, 0.0f, // Posición temporal
272     -1.0f, -1.0f, 0.0f, // Dirección hacia adelante (-X)
273     1.0f, 1.0f, 0.0f, // Atenuación
274     15.0f); // Ángulo
275 spotLightCount++;
276
277 // --- LUZ HELICÓPTERO ATRÁS (Índice 2) ---
278 spotLights[2] = SpotLight(1.0f, 1.0f, 0.0f, // Color Amarillo
279     0.0f, 2.0f, // IA, ID
280     0.0f, 0.0f, 0.0f, // Posición temporal
281     1.0f, -1.0f, 0.0f, // Dirección hacia atrás (+X)
282     1.0f, 1.0f, 0.0f, // Atenuación
283     15.0f); // Ángulo
284 spotLightCount++;
285
286 // --- LUZ AMARILLA DEL PEZ (Índice 3) ---
287 spotLights[3] = SpotLight(1.0f, 1.0f, 0.0f, // Color Amarillo
288     0.0f, 2.0f, // IA, ID
289     0.0f, 0.0f, 0.0f, // Posición temporal
290     0.0f, 0.0f, -1.0f, // Dirección inicial (Frente)
291     0.1f, 0.2f, 0.0f, // Atenuación
292     25.0f); // Ángulo
293 spotLightCount++;
294
295

```


Código Logica de las luces del faro y el pez:

```
339 // --- CICLO DE LUZ PEZ Y FARO ---
340 // Control de encendido/apagado de luces puntuales según el estado capturado en Window
341 switch (mainWindow.getLuzPrendida())
342 {
343     case 0: shaderList[0].SetPointLights(pointLights, 2); break; // Ambas encendidas
344     case 1: shaderList[0].SetPointLights(pointLights, 0); break; // Ambas apagadas
345     case 2: shaderList[0].SetPointLights(&pointLights[1], 1); break; // Solo pez (Azul)
346     case 3: shaderList[0].SetPointLights(&pointLights[0], 1); break; // Solo farola (Blanca)
347 }
```

Código Lógica luz del pez con rotación:

```
// --- LÓGICA PEZ Y LUZ AMARILLA ---
// Obtención de variables de control dinámico (Movimiento y Rotaciones)
GLfloat movPez = mainWindow.getcofre();
GLfloat rotX = mainWindow.getrotacion1();
GLfloat rotY = mainWindow.getrotacion2();
GLfloat rotZ = mainWindow.getrotacion3();
glm::vec3 posBulbo = glm::vec3(5.0f + movPez, 3.2f + movPez, -5.0f);

// Construcción de la matriz de rotación para orientar la luz según el pez
glm::mat4 rotMatPez = glm::mat4(1.0f);
rotMatPez = glm::rotate(rotMatPez, glm::radians(rotX), glm::vec3(1.0f, 0.0f, 0.0f));
rotMatPez = glm::rotate(rotMatPez, glm::radians(rotY), glm::vec3(0.0f, 1.0f, 0.0f));
rotMatPez = glm::rotate(rotMatPez, glm::radians(rotZ), glm::vec3(0.0f, 0.0f, 1.0f));

// Transformación del vector frontal (1,0,0) mediante la matriz de rotación
glm::vec3 dirLuzPez = glm::vec3(rotMatPez * glm::vec4(1.0f, 0.0f, 0.0f, 0.0f));
spotLights[3].SetFlash(posBulbo, dirLuzPez);
```

Código Lógica del helicóptero y sus luces:

```
355 // Actualización de luces del helicóptero basadas en su posición de movimiento en X
356 glm::vec3 posHeli = glm::vec3(mainWindow.getmuevex(), 3.0f, 0.0f);
357 spotLights[1].SetFlash(posHeli, glm::vec3(-1.0f, -0.8f, 0.0f));
358 spotLights[2].SetFlash(posHeli, glm::vec3(1.0f, -0.8f, 0.0f));

378 // Gestión de visibilidad de luces del helicóptero según la dirección de vuelo
379 if (mainWindow.getDireccionHeli() == 1) { // Vuelo hacia adelante
380     spotLights[2].SetFlash(glm::vec3(0, -100, 0), glm::vec3(0, -1, 0)); // Ocultar luz trasera
381     shaderList[0].SetSpotLights(spotLights, 4);
382 }
383 else if (mainWindow.getDireccionHeli() == 2) { // Vuelo hacia atrás
384     SpotLight temp = spotLights[1];
385     spotLights[1] = SpotLight(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0); // Desactivar luz delantera
386     shaderList[0].SetSpotLights(spotLights, 4);
387     spotLights[1] = temp;
388 }
389 else { // Helicóptero estático (Apagar ambas luces frontales/traseras)
390     SpotLight temp1 = spotLights[1];
391     SpotLight temp2 = spotLights[2];
392     spotLights[1] = SpotLight(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0);
393     spotLights[2] = SpotLight(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0);
394     shaderList[0].SetSpotLights(spotLights, 4);
395     spotLights[1] = temp1;
396     spotLights[2] = temp2;
397 }
```

Código instancia de la farola:

```
410 // --- FAROLA ---
411 model = glm::mat4(1.0);
412 model = glm::translate(model, glm::vec3(0.0f, 1.5f, 5.0f));
413 // Sincronizar posición de la luz puntual con el modelo visual de la farola
414 pointLights[0].SetPos(glm::vec3(model[3][0], model[3][1], model[3][2]));
415 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
416 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
417 Farola_M.RenderModel();
```

Código del helicóptero:

```
419 // --- HELICÓPTERO ---
420 model = glm::mat4(1.0);
421 model = glm::translate(model, posHeli);
422 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
423 model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
424 model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
425 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
426 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
427 Blackhawk_M.RenderModel();
```

Código Acuario:

```
429 // --- ACUARIO (Manejo de Transparencias) ---
430 model = glm::mat4(1.0);
431 model = glm::translate(model, glm::vec3(5.0f, 2.0f, -5.0f));
432 model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
433 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
434
435 // Primera pasada: Dibujar caras opacas (traseras)
436 glEnable(GL_DEPTH_TEST);
437 glDepthMask(GL_TRUE);
438 glEnable(GL_CULL_FACE);
439 glCullFace(GL_BACK);
440 Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
441 Pecera_M.RenderModel();
442
443 // Segunda pasada: Dibujar cristal con mezcla de colores (Blending)
444 glEnable(GL_BLEND);
445 glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
446 glDisable(GL_CULL_FACE); // Permitir ver a través
447 glDepthMask(GL_FALSE); // No escribir en profundidad para no bloquear objetos internos
448 glEnable(GL_POLYGON_OFFSET_FILL);
449 glPolygonOffset(-1.0f, -1.0f); // Evitar parpadeo (Z-fighting) con el modelo anterior
450 Pecera_M.RenderModel();
451
452 // Restaurar estados de renderizado estándar
453 glDisable(GL_POLYGON_OFFSET_FILL);
454 glDepthMask(GL_TRUE);
455 glEnable(GL_CULL_FACE);
456 glDisable(GL_BLEND);
```

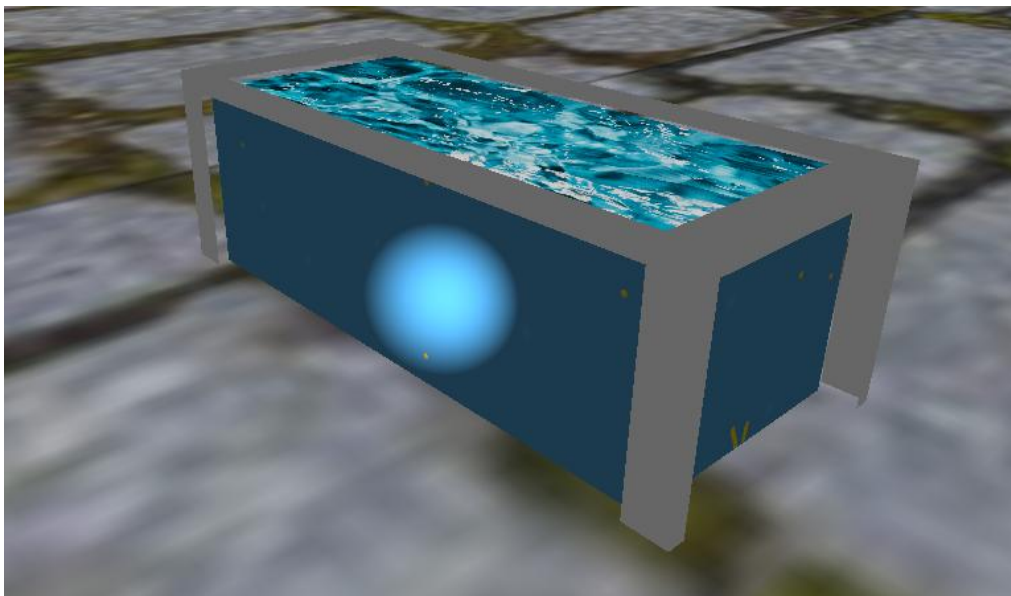
Nota: Tuve problemas para hacer que las texturas fueran traslucidas así que empleé ayuda de la IA Generativa Google Gemini IA para poder corregir mi código

Código Pez y luces:

```
458 // --- PEZ Y SUS LUCES ---
459 // Actualización de la luz puntual azul que emana del pez
460 pointLights[1] = PointLight(0.0f, 0.0f, 1.0f, 1.5f, 1.5f, posBulbo.x, posBulbo.y, posBulbo.z, 0.0f, 0.5f, 0.5f);
461
462 model = glm::mat4(1.0);
463 model = glm::translate(model, posBulbo);
464
465 // Aplicación de rotaciones de usuario al modelo
466 model = glm::rotate(model, glm::radians(rotX), glm::vec3(1.0f, 0.0f, 0.0f));
467 model = glm::rotate(model, glm::radians(rotY), glm::vec3(0.0f, 1.0f, 0.0f));
468 model = glm::rotate(model, glm::radians(rotZ), glm::vec3(0.0f, 0.0f, 1.0f));
469
470 // Ajuste de orientación y escala final del modelo importado
471 model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
472 model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
473
474 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
475 Pez_M.RenderModel();
```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

El principal error al que me afronte fue en la actividad 2 ya que si bien ya tenía mi modelo de pecera completo y este tenía el formato de texturas no contaban con canales RGB y Alpha al momento que intentaba instanciarlo inicialmente, corregí esto y posteriormente me enfrente al problema que no se veían traslucidas como yo quería (no se veía bien a través de ellas), y finalmente cuando ya había arreglado el blending ahora no se veían bien los elementos internos de la pecera así que para solucionarlo luego de intentar por 1 día completo recurrí a la IA generativa Gemini de Google, con la cual ya pude solucionar el problema y finalmente mi modelo quedo completo.



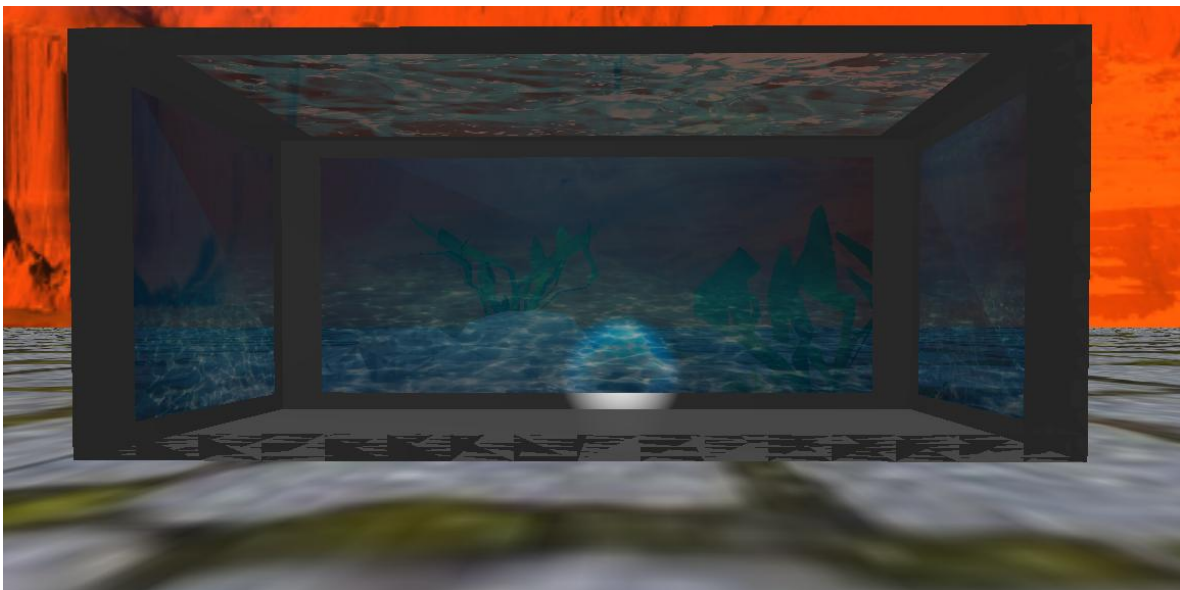
Error 1: las texturas no cargan correctamente la transparencia

Solución verificar a mano que cada textura incluya los canales RGB y Alpha



Error 2: Se generan triángulos extraños al momento de ver la transparencia

Solución configurar correctamente el blending



Error 3: Ahora no se aprecian los detalles internos de las piedras y algas de la pecera

Solución: Luego de intentar por 1 día emplee la IA Generativa Google Gemini para corregir mi código

Propmts empleados para la corrección problemas:

Yo: Hola, Necesito ayuda para completar el ejercicio de mi práctica. Estoy intentando renderizar una pecera de la cual ya tengo el modelo obj y me asegure que sus texturas sean RGB con canal alfa, mi problema es que al momento de intentar poner la transparencia tengo errores y no carga bien la imagen ya que se generan triángulos extraños y cuando no sucede esto no se ven los elementos internos de la pecera te adjunto el código que ya tengo (Archivo P08-317031814), preferentemente b́asate en el sitio <https://learnopengl.com/Advanced-OpenGL/Blending>

Google Gemini:

✦ ¡Claro que sí! Entiendo perfectamente la situación con el renderizado de transparencias en OpenGL. Basándome en los principios de **LearnOpenGL** sobre el *Blending* y el orden de dibujado, aquí tienes la explicación y el fragmento de código que necesitas para que tu pecera se vea correctamente:

Para lograr una transparencia realista en un modelo complejo como una pecera, no basta con habilitar el mezclado de colores. Es necesario realizar un proceso de dos pasadas para que el buffer de profundidad (*Z-buffer*) no descarte los fragmentos que están detrás del cristal. Siguiendo la lógica de "Blending" del sitio mencionado, el código para tu ejercicio quedaría así:

C++



```
// --- ACUARIO (Manejo de Transparencias) ---
// Definimos la posición y escala de la pecera en la escena
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(5.0f, 2.0f, -5.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

// --- PASO 1: Renderizado de Caras Sólidas ---
// Dibujamos primero las caras internas/traseras para que el buffer de profundidad
glEnable(GL_DEPTH_TEST);
glDepthMask(GL_TRUE);
glEnable(GL_CULL_FACE);
glCullFace(GL_BACK); // Descartamos las caras que no miran a la cámara
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
Pecera_M.RenderModel();
```

```

// --- PASO 2: Renderizado del Cristal (Blending) ---
// Activamos el mezclado de colores (Alpha Blending) según la fórmula estándar de
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

glDisable(GL_CULL_FACE); // Desactivamos el descarte de caras para poder ver el i
glDepthMask(GL_FALSE); // IMPORTANTE: Desactivamos la escritura en el buffer de
                        // para que los objetos dentro de la pecera (como el pez

// Aplicamos un pequeño offset para evitar el "Z-fighting" (parpadeo) entre las p
glEnable(GL_POLYGON_OFFSET_FILL);
glPolygonOffset(-1.0f, -1.0f);

Pecera_M.RenderModel();

// --- RESTAURACIÓN DE ESTADOS ---
// Es vital regresar OpenGL a su configuración original para no afectar el render
glDisable(GL_POLYGON_OFFSET_FILL);
glDepthMask(GL_TRUE);
glEnable(GL_CULL_FACE);
glDisable(GL_BLEND);

```

Con este método, primero aseguras la estructura del modelo y luego superpones la capa de transparencia sin que el sistema de profundidad de OpenGL elimine por error los píxeles de lo que esté dentro de la pecera.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

En esta ocasión la practica si me costo trabajo, tener que adaptar el apagado de luces a elementos móviles, tener que variar entre arreglos de luces y manejar elementos con transparencias me resulto muy cansado, honestamente esta practica me llevo a mi limite y por momentos no sabía si lograría acabar a tiempo, la complejidad principal se centro en manejar arreglos, que estos respondieran a acciones con teclado y además en manejar transparencias.

En general el ejercicio 1 no me costó trabajo aunque si me fue un poco pesado, en comparativa los ejercicios 2-4 me resultaron en extremo agotadores y en especial el ejercicio 2 me costó muchísimo por el manejo de “vidrio” en la pecera, llegando al punto de verme forzado a recurrir al uso de IA generativa para poder solucionarlo.

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

En referencia a detalles de la explicación teórica me pareció adecuada la explicación vista en la sesión del laboratorio no obstante en esta ocasión siento que quizá exponer un par más de ejemplos habría ayudado a que fuera más fácil realizar la práctica.

- c. Conclusión de la práctica:

De esta práctica puedo concluir que el manejo de normales, blending y arreglos de luces permite una integración de iluminación mucho más detallada en un escenario, así como que el comprender como es que ciertos materiales manejan “transparencia” permite crear efectos como los de una pecera, además pude entender que el proceso de instancia y manejo de dichas texturas es muy delicado ya que si no se hace bien puede conllevar a muchos problemas al renderizar los elementos.

Otro elemento que quisiera destacar es como el manejo de arreglos de luces permite alterar el escenario dándole complejidad con los elementos existentes, sin necesidad de instanciar constantemente múltiples luces ya que esto resultaría en un manejo ineficiente de los recursos.

Bibliografía

- *LearnOpenGL - Blending*. (n.d.). <https://learnopengl.com/Advanced-OpenGL/Blending>
- *Normales — blender manual*. (n.d.). <https://docs.blender.org/manual/es/2.93/modeling/meshes/editing/mesh/normals.html>